

The urnings algorithm

Application in Skilltrie and prospects

Skilltrie
Utrecht University
June 1, 2026

Abstract

The urnings algorithm is a robust rating engine used to assess the skill of players and the difficulty of items in the Skilltrie webapplication. This essay will go over the mechanisms of the urnings algorithm, explaining both heuristically and mathematically the dynamics of such a system. In this document, special attention will be placed on how the urnings algorithm is used in our webapplication, and how it can be iterated on in the future. Suggestions of features not yet implemented are written down in order of priority of when we believe they should be implemented.

Keywords: Urnings, Markov Chains, Bernouilli, Rating

Introduction

In recent education, adaptive learning has become quite the hot topic. Having the content and difficulty of exercises adapt to the level of a student is generally a good thing, keeping students challenged and invested, while ensuring they learn as much as possible. To aid, this, techniques in adaptive learning have been studied and implemented in digital learning environments that are widely used today.

The Urnings algorithm is a rating system developed to track how learner abilities and item difficulties change over time in such adaptive learning environments. It aims to provide a simple yet effective model, where single iterations are easy to compute, and the current state of the system has statistical significance.

Popular approaches like the Elo rating system suffer from key statistical drawbacks: they do not provide standard errors and lead to rating variance inflation. The Urnings algorithm

was designed specifically to address these shortcomings. In Urnings, every object we are tracking is represented by an urn filled with a combination of green and red balls. When two objects are placed in a "match" against each other, the amount of red and green balls may shift (though the total will always remain the same). Then, through the amount of green balls an object has, a rating can be derived. In this document, Urning will be used to refer to the rating of an Object, where the Urning is defined as the ratio of green balls inside of the urn.

Concretely for our learning platform, the objects that will be placed in "matches" are the User and the Items they have to answer inside of a level. The amount of green balls the user has represents their knowledge inside of a certain domain. The amount of green balls an Item has represents its difficulty rating. Each item will have 1 urn that represents its difficulty, and every user will have multiple urns, one for each topic, so that it represents their skill inside of a certain topic. This only holds under the assumption that topics are uncorrelated, but if they are in fact correlated, this can be circumvented by having larger urns (for domains) that also influence the rate of change of these smaller urns.

Basic update rule of the algorithm

Whenever a User answers an Item, the outcome of the "match" will be Correct or Incorrect. Both the User and the Item also have an urn representing their rating. We compare the actual outcome of the match with what we would expect from the match from the urn. Here, the winner of the match is the one who draws the green ball, and the loser is the one who draws the red ball. Balls will be drawn until a winner can be decided (i.e. only one of the players draws a green ball).

Supposing the drawn balls are bernouilli distributed, with Urning rating p_u for the user and p_i for the item. Furthermore, let u_u and u_i be the amount of green balls for user and item respectively. Let us denote the outcome of our match by the random variable $X \in \{U, I\}$, where U is the user "winning" the game, and I is the item "winning" the game. Then, we can derive that the probability for us to predict that the user wins is:

$$p(X = U | p_u, p_i) = \frac{p_u(1 - p_i)}{p_u(1 - p_i) + p_i(1 - p_u)} \quad (1)$$

Similarly, the probability for the item to win is:

$$p(X = I | p_u, p_i) = \frac{p_i(1 - p_u)}{p_u(1 - p_i) + p_i(1 - p_u)} \quad (2)$$

Furthermore, we call the true probabilities, which are unknown, π_u and π_i . The probability for a real item or user win will be the same as above, just with the probabilities $u \rightarrow \pi$.

The main update rule works by observing some result, and then making a prediction based on what we would expect the outcome to be, based on Eq. 1. If our observation and prediction

match, we don't have to update anything, but if there is a mismatch, we update the urns of the Player and Item so that there is more chance to predict our observation in the future. With this in mind, we illustrate the simples update scheme in the following flowchart:

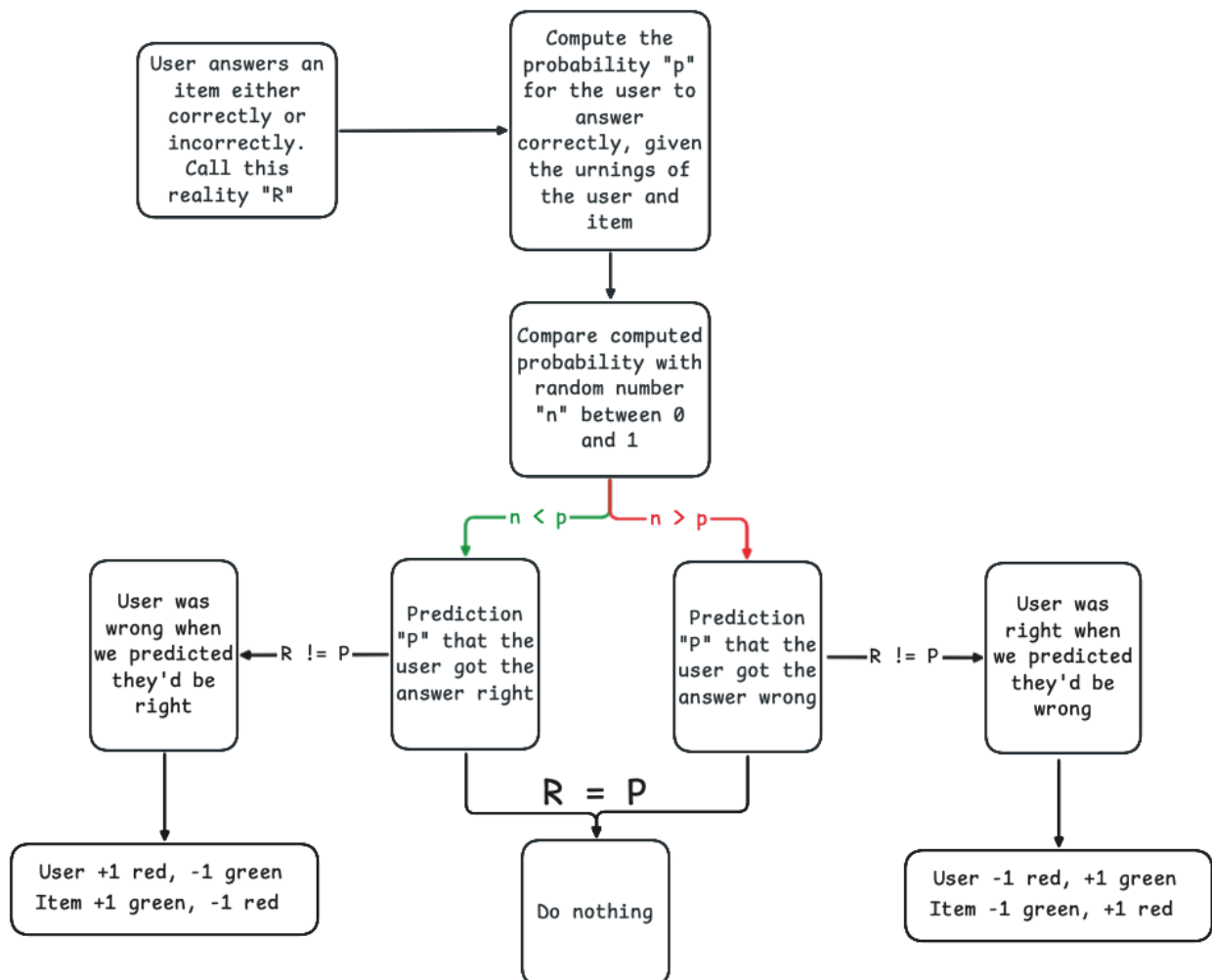


Figure 1: A flowchart showing how the urns algorithm iterates the urns of a user and item after completion of a "match"

For users, this heuristically means that surprising correct answers raise the rating, and surprising incorrect answers lower the rating. For Items, this heuristically means that surprising correct answers lower the difficulty, and surprising incorrect answers raise the difficulty.

With this, our bread and butter is set. With this simple update rule, we are able to track skill of players and difficulty of items dynamically. These updates depend only on the current state of our system, and not on the history of the rating of the item or user. Besides decreasing algorithmic complexity, this has the nice bonus that we don't need to track history in our database to say something about the skill or difficulty.

However, there is still more to gain from the urnings algorithm. For the next sections, we will be discussing the additions and applications we have implemented in our codebase, as well as some proposed future tweaks that will make the urnings algorithm nicer to work with.

Initial urn size

The amount of balls inside of an urn should always be fixed. Our update rule makes sure that for each green ball added, a red ball is taken away and vice versa. Then, the sum of green and red balls of an urn gives us our constant urn size.

The urn size is an important hyperparameter, as it dictates the rate at which change occurs. A small urn size will result in rapid changes in rating, but will increase variance between current rating and actual rating. A larger urn size, on the other hand, will cause ratings to change slowly, while giving a higher certainty that the current rating is the true one.

For each item, we will expect many users to be "matched" with it, so that we are allowed to make these urns larger, in exchange for slower convergence to true difficulty. On the other hand, users have an urn per topic, and will not be spending much time in each topic. Therefore, these urns should be smaller.

Initial values have been used inside of the codebase: 10 balls for users and 100 balls for items. Later on, it would be advisable to run A/B tests on this to better settle on a good value. Ideally, the total urn size of the user will depend on the average amount of items a user makes per topic, and the urn size of an item on the average amount of users that make a topic in a certain timespan.

User Mastery

One of the first applications that could be seen in the urnings rating is that of user mastery. Basically, the urning of a user can be seen as a persistent measure of their competence at a certain topic.

Now, it would be tempting to define mastery of a topic as the urning of a user surpassing a certain threshold value. The problem with this is that it is probability based, and can thus be

unsatisfying and un insightful for users when they simply won't progress above the threshold despite answering items correctly.

However, it is nice that their progression is not only based on individual tests, but also on their history through this urning. Suppose for example we only look at tests. Then, it could happen that a user progresses by accident because they guess the answers correctly. The urning supplies us with a non-volatile, slower rate of change that makes sure outliers don't influence progression too much.

The solution we have is that we use both the most recent test result and the urnings rating in progression. We use a weighted average of proficiency (weight 1) and urnings (weight 2) and check if this is above a certain mastery threshold. This gives us the best of both worlds. Of course, the weightings can be experimented with further to settle on a distribution that feels satisfying for the users.

Metropolis Hasting

Statistically, we want the urns to be completely independent of each other. If this is the case, the true probability π_i and π_u causes the current distribution of the two urns. For our urns to be distributed similarly to the "true" urns from the skill/difficulty of the user/item, we must have according to Theorem 1 from [?] that the distribution of the urnings of the two ratings follow:

$$p(u_u, u_i) = \frac{[u_i(n - u_u) + (n - u_i)u_u] \binom{n}{u_i} \pi_i^{u_i} (1 - \pi_i)^{n-u_i} \binom{n}{u_u} \pi_u^{u_u} (1 - \pi_u)^{n-u_u}}{Z} \quad (3)$$

Where Z is a normalization factor summing up the probability of all possible states. However, if these urnings were truly independent of each other (which they should be for the nicest theoretical properties), we would expect this probability to simply be the product of the binomials. Thus, we have a factor $[u_i(n - u_u) + (n - u_i)u_u]$ that forms a problem. As this only depends on the known values of u_i, u_u and n , we can eliminate this using a Metropolis Hastings step.

To put it simply, when we propose a change, we draw a random number between 0 and 1, and accept the change only if that value is smaller than:

$$\frac{u_i(n - u_u) + (n - u_i)u_u}{u_i^*(n - u_u^*) + (n - u_i^*)u_u^*} \quad (4)$$

Where u^* signifies the proposed amount of green balls in the urn. By selectively rejecting proposed states with this probability, our system visits each state with a frequency that exactly cancels out our unwanted factor, ensuring the stationary distribution matches the

independent binomial distribution. Because of this, we are able to assume that all urns are independent.

Matchmaking [NOT YET IMPLEMENTED]

A second important way we can use the urnings rating in our application is in matchmaking. Users will then be matched with Items that match their skill level. This way, users will be sufficiently engaged with the material, as it challenges them to a nice degree.

The main formula we use for matchmaking is that we weight each of the potential items that a user can draw from by a factor $M(p_i, p_u)$, which is the matchmaking function.

This matchmaking function only needs to meet two requirements:

1. It must be a probability distribution over the different possible matches.
2. Each match must have a positive, non-zero weight.

As long as our function meets these requirements, it is a valid matchmaking function. Usually, we would want the matchmaking probability to have some inverse relation to the difference in rating.

Matchmaking Metropolis Hasting [NOT YET IMPLEMENTED]

In case we use matchmaking, we introduce a new bias into our systems. This is because the act of matchmaking players with similar skill makes it more likely that ratings are changed in a way that the Item and User won't be matched again. (Because of similar urnings, they are likely to drift apart and the prediction will be most likely to be wrong compared to non-rating based matchmaking).

This problem can be circumvented using another Metropolis-Hastings (MH) step inside of the final change. We only accept the change of urnings with a certain probability based on if the change makes it more or less likely that the same match is made again. This step ensures that if an update makes a replay between item and user more likely (notwithstanding the fact that we limit rematches externally by filtering out repeating items). If it doesn't make replays more likely, we accept the change with a probability that depends on the likelihood of a replay.

To combat this, we update our Metropolis hastings step, but setting the acceptance chance to become:

$$\frac{M(p_u^*, p_i^*)}{M(p_u, p_i)} * \frac{u_i(n - u_u) + (n - u_i)u_u}{u_i^*(n - u_u^*) + (n - u_i^*)u_u^*} \quad (5)$$

With $M(p_u, p_i)$ the probability to match urns with urning p_u and p_i .

Variance [NOT YET IMPLEMENTED]

One of the big advantages of the Urnings algorithm is that it has a known variance for how well the urnings rating predicts the true skill. As long as we are dealing with a mixed Markov Chain, we are able to compute exactly the variance to be that of a binomial distribution, i.e.

$$\sigma^2(p_i) = np_i(1 - p_i) \quad (6)$$

Then, this can be used to give a 95% likelihood range for the rating of a user or item, so that we can reason about their skill level or difficulty more statistically.

Anomaly detection [NOT YET IMPLEMENTED]

One specific implementation of the variance lies in anomaly detection. For this, we wish to know the variance of the

1. Compute the probability that the user answers correctly $p(X = U|p_u, p_i)$ as per Eq. (1).
2. Compute the variance of the outcome, which can be derived to be

$$\sigma^2 = p(X = U|p_u, p_i)(1 - p(X = U|p_u, p_i))$$

3. compute the z-score

$$z = \frac{O - p(X = U|p_u, p_i)}{\sigma}$$

where $O \in \{0, 1\}$ is the outcome of our actual measurement.

4. Store one mark on our item if the z-score exceeds some threshold.
5. After an item has accumulated enough marks, flag it as an anomaly

With this scheme, we have a statistical way to detect anomalies. This can be used, for instance, for the detection of faulty AI-generated questions, or to prove that there is no statistical difference in how AI questions are perceived by the users.

MCMT [NOT YET IMPLEMENTED]

The system of our app forms an intractable Markov Chain. If we assume some local perturbation (adding a user, adding an item), we cannot actually assume that our distribution follows a binomial. The system has a so-called Markov Chain Mixing Time (MCMT) before this local perturbation fades and the Markov Chain is pure again. Only once the MCMT has been reached, can we accurately use the Variance of the limiting distribution of the Markov Chain, and therefore we cannot say much statistically about our system until this is the case. The MCMTs can be as large as 30 iterations for an urn of size 10, and therefore we would need at least 30 items to be done before they are mixed into the system. Of course, this results

in slow progression, and therefore we would not advice to use the variance for the player progression. For items, having about 100 balls in an urn also means that the MCMT can go up quite high, to values exceeding 250 iterations.

The way in which the MCMT would be calculated is as follows:

1. Take an array of possible skill values inside of the urnsize n , i.e. $[0, 1/n, 2/n, \dots, 1]$
2. For each skill value s , run a simulation until our terminating condition holds.
3. The simulation takes N different users at skill level s , and starts them at skill 0.5. Then, for each iteration, run a step of the urnings algorithm. Finally, check whether the distribution of our N different users matches that of a true Urnings Algorithm (as per the literature values discussed before).
4. If it does so for at least X consecutive iterations, take that first time as the MCMT for that skill level.
5. Finally, take the maximum over all skill levels as an upper bound of the MCMT. This is how long a new user would have to try before we can be certain they have mixed into our Markov Chain.

Note that this MCMT is a constant of the urnsize, as we are taking the upper bound and the actual skill is unknown. Because of this, these MCMT's have to be computed once, before they can be used as a measure to indicate whether a system has been mixed after adding a new item or user. Furthermore, by assuming that perturbations have no long-range effects (e.g., new users do not perturb other users), we can still keep the program running as it did before when adding new items or users. This assumption only holds for large systems and relatively large item urns.

One way to solve this problem of having large MCMTs is by using an intake quiz. If we can statistically determine a range of the "actual" skill of a user by having them do a more heavily weighted intake quiz, the MCMT will decrease drastically, and perturbations in the Markov Chain are less consequential.

Conclusions

To conclude, these are all methods that have been implemented or are proposed to be implemented in the future.

We suggest further research to still be done. Specifically, simulations of systems is a nice feature that could aid theoretical research and help guide descisions on these algorithms. Furthermore, a healthy dose of A/B testing would also be beneficial in determining the different hyperparameters of the system.